

35.9. BONHOMME qui marche : le faire bouger depuis principal.cpp

Le premier BONHOMME avance d'un pixel par image : `principal.cpp` change `x`, `montrerHeros()` le repose à chaque tour.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : une variable `x` pour le premier BONHOMME, qui grandit d'un pixel à chaque image, jusqu'à 120. `montrerHeros(0, x, 64)` est appelé **dans la boucle**, à chaque tour : le lutin suit.

Les trois fichiers n'ont pas bougé : `perso_BONHOMME.cpp` et `heros.cpp` sont ceux de l'étape d'avant. Seul `principal.cpp` change — il décide **quand** et **où**, les autres fichiers savent **comment**.

Déroulé : au départ `x` vaut 0 ; image 1 → `x` = 1, BONHOMME en (1, 64) ; image 2 → (2, 64) ; ... ; image 120 → (120, 64), et `if (x < 120)` devient faux : il s'arrête. Le second reste en (104, 64) : l'un passe sur l'autre.

Essaie : fais avancer le second aussi, avec une variable `x2` qui part de 104 et recule.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)
- `#include <sprite16>` → [35.2. La fonction `sprite16\(\)` — un personnage de 16 × 16](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **504 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	50 o	← la console, à l'allumage — la mise en route de la console
montrerHeros()	219 o	← main() — ta fonction
main()	52 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
Tuiles	64 o	← CopierTuiles — 0 lettre sur 43, et 4 tuiles dessinées
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

<code>sprite()</code>	x 4	218 o sur place, dans <code>montrerHeros()</code>
<code>image()</code>	x 1	6 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)
<code>sprite16()</code>	x 1	0 o sur place, dans <code>montrerHeros()</code>

Tes fonctions appelées :
montrerHeros() x 2 29 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// principal.cpp fait marcher BONHOMME : x change, montrerHeros() suit.  
  
#include "personnages.cpp" // la liste : BONHOMME  
#include "heros.cpp"       // montrerHeros(numero, x, y)
```

2. UNE VARIABLE — X

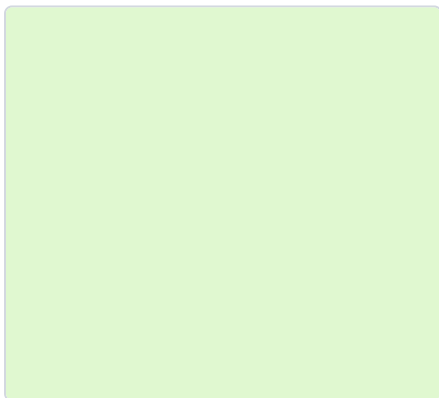
```
uint8_t x = 0; // la place du premier BONHOMME, en pixels : de 0 à 120
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

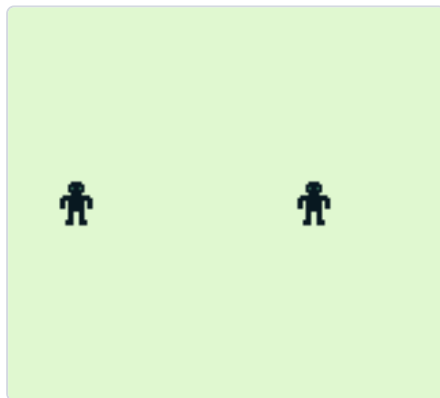
```
int main() {  
    montrerHeros(4, 104, 64); // le second, posé une fois : il ne bouge pas  
  
    while (true) {  
        image(); // une image : 1/60 de seconde  
        if (x < 120) x = x + 1; // un pixel de plus, jusqu'à 120  
        montrerHeros(0, x, 64); // le premier, reposé à sa nouvelle place  
    }  
}
```

3. Ce qui se passe, étape par étape

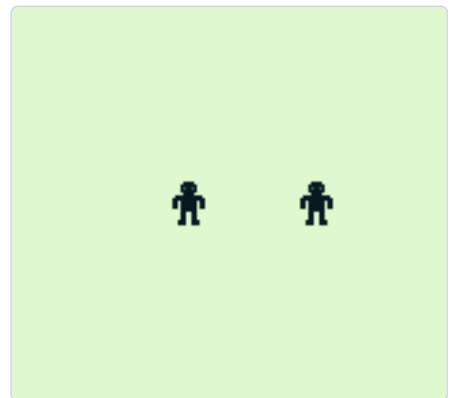
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



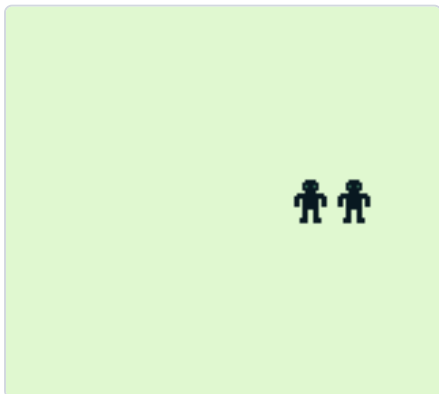
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le premier bonhomme traverse l'écran vers la droite et s'arrête ; le second ne bouge pas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ il avance : au bout d'une demi-seconde, il a quitté le bord ($x = 47$)

✓ puis il s'arrête en 120

✓ le second n'a pas bougé

Ensuite : **35.10. BONHOMME verrouillé : on s'en sert, on ne le modifie pas** — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++